Universus

Dokumentacja techniczna kodu

Przeglądarkowa bijatyka 2D z deterministyczną symulacją i rollback netcode

Co robi każda część kodu, dlaczego istnieje i jak wszystko uruchomić.

Stan repozytorium: listopad 2026

Spis treści

1. Czym jest Universus **3**

1.1 Najważniejsze pojęcia . . . 3

2. Architektura **4**

2.1 Stos technologiczny i powody wyboru . . . 4

2.2 Niezmienniki (zasady, których nie wolno łamać) . . . 4

2.3 Formaty danych wspólne dla wszystkich części . . . 5

3. Struktura repozytorium **6**

4. Symulacja (sim/) **7**

4.1 Jak działa wybór ruchu . . . 8

4.2 Combosy, ruchy wieloetapowe i wielokrotne trafienia . . . 8

4.3 Rollback krok po kroku . . . 8

5. Dane postaci i balansu (data/) **9**

5.1 Pola ruchu . . . 9

6. Klient (client/) **10**

7. Serwer (server/) **12**

7.1 Migracje . . . 12

7.2 Endpointy . . . 12

8. Narzędzia (tools/) **13**

9. Testy i CI **14**

10. Instalacja i uruchomienie **15**

10.1 Wymagania . . . 15

10.2 Pierwsze uruchomienie (gra lokalna) . . . 15

10.3 Gra przez sieć w trybie deweloperskim . . . 15

10.4 Baza danych (opcjonalnie) . . . 15

10.5 Pełna wersja z jednego portu (jak na produkcji) . . . 15

10.6 Testy lokalnie . . . 15

10.7 Parametry URL . . . 16

10.8 Sterowanie . . . 16

11. Typowe zadania **17**

11.1 Dodanie lub zmiana ruchu . . . 17

11.2 Dodanie postaci . . . 17

11.3 Dodanie grafiki . . . 17

11.4 Po znalezieniu desync . . . 17

12. Stan projektu **18**

1. Czym jest Universus

Universus to bijatyka 2D (gatunek Street Fightera) działająca w przeglądarce, napisana jako praca inżynierska. Dwóch graczy walczy na jednym ekranie: ciosy, bloki, rzuty, ataki specjalne, combosy, rundy. Grać można lokalnie na jednej klawiaturze, przeciwko AI, w trybie treningowym albo przez sieć.

Najważniejsza część projektu to nie sama gra, tylko to, co ją wspiera: **deterministyczna symulacja** (ten sam ciąg wejść zawsze daje bit w bit ten sam stan gry) i **rollback netcode** (gra przez sieć bez czekania na pakiety; błędne przewidywanie jest cofane i odgrywane ponownie). Praca mierzy, jak dobrze to działa: czas klatki, głębokość rollbacku, częstość błędnych przewidywań.

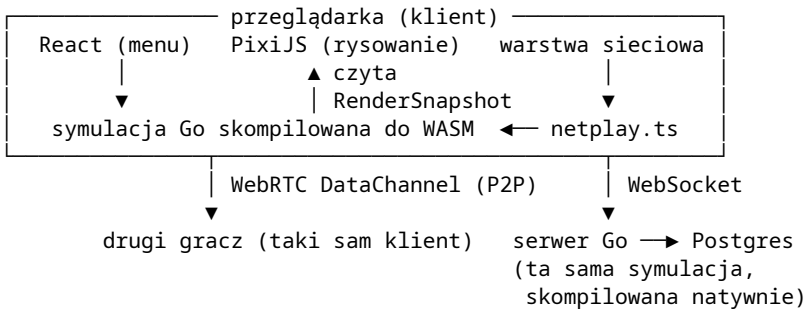
Zasada nadrzędna projektu: **praca dyplomowa jest ważniejsza niż artefakt**. Jeśli trzeba coś obciąć, obcina się zawartość (postacie, grafikę), nigdy testy, pomiary ani instrumentację.

1.1 Najważniejsze pojęcia

Pojęcie	Znaczenie
Klatka (frame)	Jeden krok symulacji. Gra działa ze stałą częstotliwością 60 klatek na sekundę. Numer klatki to jedyny zegar symulacji.
Determinizm	Te same wejścia w tej samej kolejności zawsze dają identyczny stan, na każdym komputerze i w każdej kompilacji (natywnej i WASM).
Rollback	Gra nie czeka na wejście przeciwnika: przewiduje je (powtarza ostatnie), a gdy prawdziwe wejście przychodzi i różni się, cofa stan i odgrywa klatki ponownie.
Fixed-point 16.16	Liczby z przecinkiem zapisane jako liczby całkowite `int32` (16 bitów części ułamkowej). Zastępują float, bo float może dawać różne wyniki na różnych platformach.
GameState	Jedna płaska struktura z całym stanem gry. Zapis i odczyt stanu to zwykła kopia bajtów.
Checksum	Skrót FNV-1a liczony z bajtów stanu co klatkę. Dwie maszyny porównują checksumy i tak wykrywają rozjazd (desync).
Frame data	Liczby opisujące ruch: startup (klatki do trafienia), active (klatki z hitboxem), recovery (klatki powrotu), hitstun, blockstun itd.
Hitbox / hurtbox / pushbox	Prostokąty: hitbox zadaje trafienie, hurtbox je przyjmuje, pushbox nie pozwala postaciom na siebie wejść.
SOCD	Jednoczesne przeciwne kierunki (lewo+prawo, góra+dół). Lewo+prawo daje neutral, góra wygrywa z dołem.
Cancel	Przerwanie ruchu, który trafił, innym ruchem (np. normal special), zanim skończy się jego recovery.
Motion	Sekwencja kierunków przed przyciskiem, np. QCF ( ) dla fireballa, DP ( ) dla uppercuta.

2. Architektura

System ma trzy części, które dzielą jeden kod symulacji:



- **Symulacja (`sim/`)** to czysty Go bez zależności. Kompiluje się do WebAssembly dla przeglądarki i natywnie dla serwera. Jeden kod oznacza, że serwer może odtworzyć mecz bit w bit i sprawdzić wynik.
- **Klient (`client/`)** w TypeScript: React tylko dla menu, PixiJS do rysowania, własna warstwa sieciowa z rollbackiem. Widok tylko czyta stan i nigdy go nie zmienia.
- **Serwer (`server/`)** w Go: łączy graczy (pokoje i matchmaking), przekazuje pakiety, gdy P2P nie działa (relay), obsługuje konta, ranking i weryfikację meczów przez ponowną symulację.
- **Dane (`data/`)**: postacie i balans w JSON, wbudowane w binarkę (`go:embed`), więc hash danych jest częścią tożsamości symulacji.

2.1 Stos technologiczny i powody wyboru

Element	Wybór	Dlaczego
Symulacja	Go WASM	Ten sam kod w przeglądarce i na serwerze. Serwer może weryfikować mecze ponowną symulacją bez osobnej implementacji (D6).
Liczby	Fixed-point 16.16 w `int32`	Float daje różne wyniki zależnie od platformy i kompilatora, co psuje determinizm. Liczby całkowite są identyczne wszędzie.
Widok	PixiJS 8 + TypeScript	Szybkie rysowanie 2D w WebGL. React tylko do menu.
Sieć	Rollback (parametry GGPO)	2 klatki opóźnienia wejścia, maks. 8 klatek rollbacku, przewidywanie = powtórz ostatnie wejście.
Transport	WebRTC DataChannel	P2P, kanał nieuporządkowany i bez retransmisji. Każdy pakiet niesie ostatnie ~8 klatek wejść, więc zgubiony pakiet nie wymaga ponownego wysłania.
Fallback	Relay przez WebSocket	Gdy WebRTC nie połączy się w 5 s, mecz idzie przez serwer. Zastępuje serwer TURN (D15).
Backend	Go + Postgres	Jeden plik binarny, migracje SQL uruchamiane przy starcie. Tylko 4 zależności Go.
Narzędzia klienta	Vite, pnpm, oxlint, vitest	Szybki dev server, lint i testy.

2.2 Niezmienniki (zasady, których nie wolno łamać)

- **Brak liczb zmiennoprzecinkowych w `sim/`.** CI przeszukuje katalog pod kątem `float32`, `float64` i pakietu `math` i przerywa build.
- **Cały stan gry jest w `GameState`.** Co nie jest w strukturze, nie cofa się przy rollbacku, a to oznacza desync. Dotyczy to też kamery, RNG, historii wejść, stanu AI, timerów i przejść rund.
- **Widok nigdy nie zapisuje stanu symulacji.** Czyta `RenderSnapshot` i rysuje.
- **Żadnych wartości gameplay w kodzie.** Frame data, obrażenia, boxy, prędkości i koszty są w JSON.
- **Symulacja jest jednowątkowa, bez zegara i bez I/O.** Jedynym zegarem jest numer klatki.

- **Bez iteracji po mapach, porównywania wskaźników, losowości bez ziarna i goroutines** w kodzie symulacji.
- **Ustawienia gracza (np. klawisze) są rozwiązywane po stronie klienta** zanim powstanie bitfield wejścia.

2.3 Formaty danych wspólne dla wszystkich części

Wejście: jeden `uint16` na gracza na klatkę

```
bit 0 góra   bit 1 dół   bit 2 lewo   bit 3 prawo
bit 4 LP     bit 5 MP    bit 6 HP     (bit 7 zarezerwowany)
bit 8 LK     bit 9 MK    bit 10 HK    (bity 11-15 zarezerwowane)
```

Te same bajty trafiają do symulacji, do sieci, do logu powtórki i na serwer.

Pakiet sieciowy

```
[uint8 typ][uint32 startFrame][uint8 count][uint16 inputs[count]]   (~20 bajtów)
typy: 1 wejścia, 2 checksum, 3 ping, 4 control
```

Każdy pakiet z wejściami niesie ostatnie ~8 klatek. Ta redundancja zastępuje retransmisję.

Kolejność kroku symulacji (część specyfikacji)

1. wejścia: SOCD → bufor → rozpoznanie motion
2. maszyny stanów postaci
3. ruch i grawitacja
4. pushboxy i granice sceny
5. pociski
6. wykrywanie trafień: najpierw hitboxy P1 vs hurtboxy P2, potem odwrotnie
7. rozstrzygnięcie trafień: obrażenia, skalowanie, hitstun, meter
8. timery, zasoby, stan rundy
9. zwiększenie numeru klatki, checksum

Kolejność jest stała, bo wymiany ciosów (trade) muszą rozstrzygać się identycznie na obu maszynach. Obrażenia też mają stałą kolejność: `base → ×starterScale/100 → ×comboScale[n]/100 → max(minDamage) → ×counterHit/100`, wszystkie dzielenia po mnożeniach, wszystko na liczbach całkowitych.

3. Struktura repozytorium

Katalog	Zawartość
`sim/`	Deterministyczny rdzeń gry, zero zależności. `sim/wasm/` to punkt wejścia WASM, `sim/replaylog/` to format logu wejść.
`data/`	JSON postaci (`data/characters/NN-nazwa.json`) i balansu (`balance.json`) oraz loader z walidacją (`data.go`).
`server/`	Backend Go: pokoje, relay, matchmaking, konta, wyniki meczów, ranking, weryfikacja, migracje SQL.
`client/`	Klient TypeScript: Vite, React (menu), PixiJS (widok), warstwa sieciowa, testy vitest.
`tools/`	Narzędzia: `replay` (powtórki i checksumy), bramka determinizmu, benchmark, generator sprite'ów.
`testdata/`	Korpus logów wejść. Każdy znaleziony błąd desync trafia tu jako log regresyjny.
`art/`	Ręcznie rysowane eksporty z Aseprite, osobny podkatalog na postać (tworzony, gdy powstaje grafika).
`.github/workflows/`	Jeden workflow CI: lint, testy, bramka determinizmu, wydajność, build klienta.

To monorepo z jednym modułem Go (`universus`). Kod Go to około 19 tys. linii w `sim/`, `server/`, `data/` i `tools/`. Klient to około 45 plików TypeScript.

4. Symulacja (sim/)

Pakiet `sim` to cała logika gry. Nie wie nic o przeglądarce, sieci ani czasie. Dostaje dwa bitfieldy wejścia na klatkę (`GameState.Advance`) i zmienia stan. Dzięki temu da się go uruchomić w przeglądarce, na serwerze, w testach i w narzędziu do powtórek, zawsze z identycznym wynikiem.

Plik	Co robi	Dlaczego jest potrzebny
`fix.go`	Typ `Fix` (16.16 w `int32`), mnożenie i dzielenie przez `int64`.	Zastępuje float. Bez tego wyniki różniłyby się między WASM a natywnym kodem.
`state.go`	`GameState` (cały stan), stałe bitów wejścia, `Advance` (9 kroków klatki), wykrywanie i rozstrzyganie trafień.	Serce symulacji. Płaska struktura bez wskaźników, więc zapis stanu to kopia.
`input.go`	Kierunki w notacji numpad, historia wejść (pierścień w stanie), rozpoznawanie motion (QCF, QCB, DP, podwójne, HCF), bufor wciśnięć.	Zamienia surowe wejście na intencję gracza. DP ma priorytet nad QCF, a skan idzie wstecz.
`fighter.go`	Maszyna stanów postaci: idle, chód, skok, dash, atak, hitstun, blok. Wybór ruchu (`moveFor`), okna cancel, follow-upy, spóźniony drugi przycisk (rzut/EX).	Decyduje, co postać robi w każdej klatce. Widok tylko to odczytuje.
`character.go`	Struktury danych postaci: `Move`, `Keyframe`, `Box`, kategorie cancel, globalny roster.	Dane niezmienne, ładowane raz. Dodanie postaci nie wymaga zmian w kodzie.
`damage.go`	Skalowanie obrażeń: starter, numer trafienia w combo, minimum, counter hit.	Bez skalowania zapętlone combo zabija. Kolejność działań jest częścią specyfikacji.
`knockback.go`	Odrzut obrońcy, juggle (trafienia w powietrzu), rosnąca grawitacja przy kolejnych trafieniach.	Combo w powietrzu samo się kończy, zamiast trwać w nieskończoność.
`projectile.go`	Pociski jako osobne obiekty w `GameState` (maks. 4).	Fireball żyje dłużej niż ruch, który go wystrzelił, więc musi być stanem i cofać się przy rollbacku.
`throw.go`	Rzuty i tech (ucieczka z rzutu).	Bez rzutu blok wygrywa ze wszystkim. Rzut sprawia, że blokowanie jest decyzją.
`resource.go`	Dwa zasoby: Drive (na rundę) i Super (na mecz) oraz struktura balansu.	System zasobów z projektu gry. Wartości liczbowe przychodzą z `balance.json`.
`round.go`	Rundy: best of 3, 99 s na rundę, KO, ekran wyniku, intro.	Przejścia rund to stan symulacji, więc też się cofają i muszą być deterministyczne.
`events.go`	Flagi zdarzeń w stanie (trafienie, blok, super…) dla efektów audio i wizualnych.	Bez tego dźwięk trafienia zagrałby 8 razy podczas rollbacku. Widok odpala efekty tylko dla potwierdzonych klatek.
`rand.go`	Generator xorshift32 ze stanem w `GameState`.	Losowość musi się cofać i powtarzać tak samo jak reszta stanu.
`checksum.go`	FNV-1a nad bajtami stanu.	Wykrywanie desync: obie maszyny porównują checksumy co 30 klatek.
`rollback.go`	`Session`: pierścień 12 zapisanych stanów, cofanie do 8 klatek wstecz.	Mechanizm rollbacku po stronie symulacji, alokowany raz.
`snapshot.go`	`RenderSnapshot`: binarny, stały układ bajtów ze wszystkim, czego potrzebuje widok.	`GameState` nigdy nie przechodzi przez granicę WASM. Widok czyta tylko snapshot przez jeden `DataView`.
`ai.go`	Przeciwnik skryptowy (poziomy easy, normal, hard) i `AIDevice`.	AI generuje tylko bitfield wejścia, dokładnie jak gracz. Nie ma żadnego uprzywilejowanego dostępu.
`dump.go`	Wypisanie każdego pola stanu.	Diagnostyka: gdy checksumy się rozjadą, pokazuje, które pole się różni.

Plik	Co robi	Dlaczego jest potrzebny
`replaylog/log.go`	Format logu wejść: nagłówek z magic `UNIV`, wersją, trybem i postaciami, potem pary `uint16` na klatkę.	Ten sam format zapisuje klient, czyta narzędzie `replay` i weryfikator na serwerze.
`wasm/main.go`	Punkt wejścia WASM: ładuje dane, trzyma sesję i bufor snapshotu w pamięci liniowej, wystawia funkcje dla JS.	Most między przeglądarką a symulacją. Przez granicę przechodzą tylko liczby i bajty snapshotu.

4.1 Jak działa wybór ruchu

Co klatkę `fighter.go` sprawdza, czy gracz może działać. Jeśli tak, `moveFor` przegląda listę ruchów postaci i wybiera ten, którego przycisk wciśnięto najpóźniej. Przy remisie wygrywa wyższa kategoria: super > Drive (EX) > special > rzut > normal. Wciśnięcie żyje w buforze przez kilka klatek, więc przycisk naciśnięty trochę za wcześnie (np. w czasie hitstopu) i tak zadziała w pierwszej możliwej klatce.

Ruch, który trafił (albo został zablokowany), otwiera **okno cancel** od pierwszej aktywnej klatki. W tym oknie może wyjść tylko ruch z kategorii wymienionej w polu `cancel` (np. `special`, `super1`). Ruch, który chybił, nie daje cancela (D74), bo inaczej każdy cios byłby bezpieczny.

4.2 Combosy, ruchy wieloetapowe i wielokrotne trafienia

Te mechaniki są opisane w danych, nie w kodzie:

- **Target combo** (np. LP > LK > MP u Kaia, HP > HP u Torva): kolejny cios to osobny ruch z polem `"after"`. Wychodzi tylko w oknie cancel poprzednika, a poza nim ten sam przycisk daje zwykły normal (D119).
- **Drugi etap specjala** (np. 236P, potem 6+LP rzuca drugi fireball): też ruch z `"after"`, a do tego `"dir"` (wymagany kierunek), `"needsHit": false` (może wyjść po chybieniu) i opcjonalne `"window"` (zakres klatek poprzednika) (D121).
- **Wiele trafień w jednym ruchu** (np. 214HP Torva): keyframe z hitboxem ma `"hitNum"`. Każdy numer trafia raz, a `HasHit` pamięta numer ostatniego trafienia (D120).

4.3 Rollback krok po kroku

- Klient zapisuje stan po każdej klatce w pierścieniu 12 stanów.
- Brakujące wejście przeciwnika jest przewidywane jako powtórzenie ostatniego.
- Gdy przychodzi prawdziwe wejście i różni się od przewidzianego, stan wraca do klatki, w której była różnica, i wszystkie klatki są liczone ponownie (do 8).
- Klatka z rollbackiem 8 klatek uruchamia symulację 9 razy i nadal musi zmieścić się w 16,6 ms. Zmierzone p99 to 0,540 ms.
- Co 30 klatek klienci wymieniają checksumy. Różnica oznacza desync i jest raportowana.

5. Dane postaci i balansu (data/)

Każda postać to jeden plik `data/characters/NN-nazwa.json`. Pliki są sortowane po nazwie, a pozycja na liście to indeks postaci w rosterze (stąd prefiks liczbowy). Obecnie są dwie postacie: Kai (`00-shoto.json`, postać wszechstronna z fireballem i uppercutem) i Torv (`01-grappler.json`, wolniejszy grappler z rzutami komendowymi).

`data.go` czyta JSON, sprawdza każde pole (ujemne klatki, hitbox poza aktywnymi klatkami, nieznany przycisk, super bez motion itd.) i zamienia wartości na struktury symulacji w fixed-point. Błąd w danych zatrzymuje start z komunikatem wskazującym pole, zamiast ujawnić się w meczu.

Pliki są wbudowane w binarkę przez `go:embed`. `Version()` liczy hash surowych bajtów, a klienci porównują go przed meczem. Dwie kompilacje z różnymi danymi odmówią wspólnej gry, bo i tak by się rozjechały.

5.1 Pola ruchu

Pole	Znaczenie
`id`	Nazwa ruchu w notacji numpad, np. `5LP`, `236HP`. Jest też nazwą tagu animacji (D113).
`input`	`stance` (stand, crouch, air), `button` (np. `LP`, `LP+LK`), `motion` (qcf, qcb, dp, qcfx2, qcbx2, hcf).
`startup`, `active`, `recovery`	Klatki do pierwszego trafienia, klatki z hitboxem, klatki powrotu.
`damage`, `hitstun`, `blockstun`, `hitstop`	Obrażenia i czasy ogłuszenia oraz zamrożenia po trafieniu.
`attackLevel`	mid, high, low, overhead (decyduje, jak trzeba blokować).
`cancel`	W co ruch może być anulowany po trafieniu: `chain`, `special`, `super1`, `super2`, `super3`.
`boxes`	Keyframe'y z hurtboxami i hitboxami `[x, y, szer, wys]`, opcjonalnie `hitNum`.
`super`, `drive`, `armor`	Poziom supera (koszt w paskach), koszt Drive (EX), liczba trafień pochłanianych przez pancerz.
`launch`, `knockback`, `juggleLimit`	Prędkość nadana atakującemu, odrzut obrońcy, limit trafień w powietrzu.
`invuln`, `landing`, `knockdown`, `throw`	Okno nietykalności, recovery po wylądowaniu, powalenie, rzut.
`projectile`	Pocisk: prędkość, czas życia, punkt pojawienia się, box.
`after`, `needsHit`, `dir`, `window`	Follow-upy: po jakich ruchach, czy wymagają trafienia, wymagany kierunek, okno klatek.
`_comment`	Komentarz dla człowieka, ignorowany przez loader.

`balance.json` trzyma wartości wspólne dla wszystkich postaci, w sekcjach: `damage` (skalowanie combo, startery, counter hit), `super`, `drive`, `round` (czasy rund), `input` (bufor), `throw`, `stage` (scena i kamera), `knockdown`, `knockback`, `juggle` i `ai`.

6. Klient (client/)

Klient to aplikacja Vite. `src/main.tsx` wybiera stronę: zwykle `App`, a dla `?sweep` stronę pomiarową. React rysuje tylko menu i panele, a gra działa na canvasie PixiJS.

Plik	Co robi	Dlaczego
`App.tsx`	Uruchamia przebieg gry według ustawień (tryb, postacie), zapisuje log wejść, łączy panel sieciowy z grą.	Spina menu, grę i sieć w jeden przebieg.
`Menu.tsx`	Ekran tytułowy i wybór postaci.	Tryb i para postaci to jedyne decyzje potrzebne do startu.
`setup.ts`	Tryby (local, ai, training, online) i odczyt ustawień z parametrów URL.	Każdy przebieg da się opisać linkiem, co ułatwia pomiary i CI.
`sim/wasm.ts`	Ładuje WASM, czyta `RenderSnapshot` przez jeden `DataView`, odtwarza go po wzroście pamięci.	Jedyny most do symulacji po stronie TS.
`game/game.ts`	Główna pętla: zegar 60 Hz, odczyt wejść, krok symulacji lub netplay, rysowanie sprite'ów, HUD i nakładki boxów.	Wszystko, co widać na ekranie podczas walki.
`game/clock.ts`	Akumulator stałego kroku.	Gra działa w 60 Hz niezależnie od odświeżania monitora (144 Hz nie przyspiesza gry).
`game/input.ts`	Klawiatura i gamepad bitfield, rozwiązanie SOCD.	Ustawienia gracza rozwiązują się tutaj, przed symulacją.
`game/sprites.ts`	Czyta eksport Aseprite i wybiera klatkę animacji na podstawie snapshotu.	Animacja wynika ze stanu, a nie z osobnego licznika w widoku.
`game/events.ts`	Odpala efekty tylko dla zdarzeń z potwierdzonych klatek.	Brak wielokrotnych dźwięków przy rollbacku.
`game/lab.ts`, `dummy.ts`	Tryb treningowy: frame advantage, wyświetlanie wejść, sterowalny manekin.	Narzędzia do nauki i testowania frame data.
`game/bot.ts`	Deterministyczne źródło wejść z ziarnem.	Pomiary rollbacku wymagają przewidywalnych, zmiennych wejść.
`game/log.ts`	Zapis logu wejść w formacie `replaylog`.	Każda sesja gry daje log regresyjny do `testdata/`.
`game/stats.ts`	Okno próbek czasu i percentyle.	Raportowanie rozkładów i p99, nigdy samej średniej.
`game/sweep.ts`, `Sweep.tsx`	Pomiar czasu klatki przy wymuszonym rollbacku 0–8 (`?sweep`).	Pomiar M-A do pracy.
`net/packet.ts`	Kodowanie i dekodowanie pakietów.	Wspólny format sieciowy.
`net/peer.ts`	Połączenie WebRTC DataChannel.	Transport P2P.
`net/room.ts`	Sygnalizacja i relay przez WebSocket do serwera, fallback po 5 s.	Działa też w sieciach, gdzie P2P nie przechodzi.
`net/netplay.ts`	Rollback netcode: historia wejść obu graczy, przewidywanie, cofanie, synchronizacja czasu, wymiana checksum, statystyki.	Rdzeń gry sieciowej.
`net/impair.ts`	Sztuczne opóźnienie, jitter i utrata pakietów.	Pomiary w kontrolowanych warunkach sieci.
`net/NetPanel.tsx`	Panel sieciowy z macierzą pomiarową (opóźnienie × utrata).	Pomiar M-B jednym kliknięciem na komórkę.
`net/account.ts`, `SignIn.tsx`	Logowanie i rejestracja, token.	Potrzebne tylko do gry online.

Plik	Co robi	Dlaczego
`net/report.ts`	Wysłanie wyniku meczu i logu na serwer.	Weryfikacja meczu po stronie serwera.

Testy klienta (`*.test.ts`) obejmują m.in. format pakietów, netplay, zegar, sprite'y i **bot-vs-bot**: dwa klienty z prawdziwymi WASM-ami grają przez symulowaną sieć (100 ms, jitter 15 ms, 5% utraty) i muszą mieć zgodne checksumy.

7. Serwer (server/)

Jeden plik binarny Go. **Baza danych jest opcjonalna**: bez `DATABASE_URL` serwer działa jako pokój i relay (tryby offline i gra po kodzie pokoju). Z bazą włącza konta, matchmaking, ranking i weryfikację.

Plik	Co robi	Dlaczego
`main.go`	Flagi `-addr` i `-static`, pokoje (`/ws?room=KOD`), kopiowanie wiadomości między graczami, start reszty usług przy obecnej bazie.	Ta sama ścieżka obsługuje sygnalizację WebRTC i relay. Serwer nie parsuje ruchu, tylko go kopiuje.
`queue.go`	Matchmaking `/ws/queue?mode=ranked|casual`, dobór po LP i RTT.	Gra z obcymi bez wymiany kodów.
`auth.go`	Rejestracja, logowanie (bcrypt), token JWT, `/api/profile`.	Ranking wymaga tożsamości. Token jest wysyłany jako pierwsza ramka WebSocket, nie w URL.
`users.go`	Dostęp do tabeli `users` bez hasha hasła.	Hash hasła nie może przypadkiem trafić do JSON.
`match.go`	`POST /api/match/{id}/result`: wyniki od obu klientów, check 1 (zgodność logów), aktualizacja rankingu.	Oba klienty raportują niezależnie, serwer porównuje.
`verify.go`	Workery weryfikacji: ponowna symulacja logu i porównanie wyniku (check 2).	Najmocniejszy argument za GoWASM: serwer liczy mecz tym samym kodem.
`plausibility.go`	Check 3: czy wejścia wyglądają na ludzkie (np. reakcje szybsze niż człowiek).	Wykrywa boty grające zgodnie z zasadami. Tylko flaguje konto do przeglądu, nigdy nie banuje.
`unilateral.go`	Rozliczenie meczu, który zgłosiła tylko jedna strona, po okresie karencji.	Wyjście z gry nie może unieważniać przegranego meczu (D52).
`rating.go`	Punkty LP i progi rang, tylko liczby całkowite.	Ranking musi być identyczny i czytelny.
`migrate.go`	Własny runner migracji: numerowane pliki `.sql`, tabela zastosowanych migracji.	Brak frameworka migracji. Kontener, który wystartował, ma aktualny schemat.

7.1 Migracje

Plik	Zawartość
`001_init.sql`	Schemat: `users`, `ratings`, `matches`, `match_submissions` (zgłoszenia obu klientów), `verification_jobs` (kolejka weryfikacji).
`002_match_facts_arrive_late.sql`	Fakty o meczu mogą przyjść po jego zakończeniu.
`003_record_what_the_ladder_paid.sql`	Zapis, ile LP mecz faktycznie zmienił.
`004_disconnects.sql`	Obsługa rozłączeń.

7.2 Endpointy

```
GET  /ws?room=KOD                 pokój: para graczy, sygnalizacja i relay
GET  /ws/queue?mode=ranked|casual matchmaking (tylko z bazą)
POST /api/auth/register           {email, password, displayName} -> {token, user}
POST /api/auth/login              {email, password}              -> {token, user}
GET  /api/profile                 -> {user}
POST /api/match/{id}/result       wynik meczu i log wejść
GET  /                            zbudowany klient (gdy podano -static)
```

8. Narzędzia (tools/)

Narzędzie	Co robi	Dlaczego
`tools/replay`	`replay gen N plik` tworzy log wejść, `replay run plik` wypisuje checksum każdej klatki, `replay dump plik klatka` wypisuje wszystkie pola stanu.	Uruchamianie symulacji natywnie bez przeglądarki.
`tools/determinism.sh`	Buduje WASM, puszcza każdy log z `testdata/` natywnie i w WASM (Node) i porównuje checksumy klatka po klatce. Przy rozjeździe pokazuje różnicę stanu.	**Bramka determinizmu**, najważniejszy test projektu.
`tools/wasm-replay.mjs`, `wasm-load.mjs`	Uruchamiają WASM pod Node i wypisują checksumy w tym samym formacie co `replay run`.	Druga strona porównania determinizmu.
`tools/bench.mjs`	Mierzy czas klatki i klatki z pełnym rollbackiem w WASM (percentyle).	Regresja wydajności: progi M0 to 0,5 ms i 4 ms.
`tools/sprites.py`	Generuje zastępcze arkusze sprite'ów i tabelę ruch tag, nakłada na nie rysunki z `art/`.	Arkusz zawsze pasuje do rostera, a niedokończona grafika i tak się ładuje.

9. Testy i CI

Jeden workflow GitHub Actions (`.github/workflows/`), działający w minutach:

```
1. go vet ./...
2. zakaz floatów: grep float32|float64|"math" w ./sim/
3. sprite'y zgodne z rosterem: tools/sprites.py + brak zmian w JSON
4. go test ./...            (w tym test poprawności rollbacku)
5. tools/determinism.sh     (natywny vs WASM, cały korpus testdata/)
6. node tools/bench.mjs     (regresja wydajności)
7. pnpm install, pnpm lint  (oxlint)
8. pnpm wasm
9. pnpm exec vitest run     (w tym bot-vs-bot, BOT_SEEDS=3)
10. pnpm build
```

- **Test poprawności rollbacku**: 100 klatek, zapis checksumu, potem te same wejścia z wymuszonymi rollbackami o głębokości 1–8 w 20 losowych (z ziarnem) klatkach. Końcowy checksum musi być identyczny. Test od razu wykrywa stan trzymany poza `GameState`.
- **Testy jednostkowe** tam, gdzie poprawność to wzór: fixed-point, kolejność skalowania obrażeń, każdy motion, każda kombinacja SOCD, krawędzie przecięć boxów, progi LP.
- **Testy na prawdziwym rosterze** (`data/data_test.go`): każdy target combo musi się połączyć, każdy ruch wielotrafieniowy musi trafić wszystkimi trafieniami, każdy drugi etap musi wyjść po chybieniu.
- **Testy z bazą** w `server/` uruchamiają się tylko z ustawionym `TEST_DATABASE_URL`.

10. Instalacja i uruchomienie

10.1 Wymagania

Narzędzie	Wersja	Do czego
Go	1.26	Symulacja, serwer, narzędzia, build WASM
Node.js	24	Klient, testy, bramka determinizmu, benchmark
pnpm	11	Zależności klienta
Python + Pillow	3.13	Tylko `tools/sprites.py`
PostgreSQL	16	Opcjonalnie: konta, ranking, weryfikacja
Docker	dowolna	Opcjonalnie: najprostszy sposób na Postgresa

10.2 Pierwsze uruchomienie (gra lokalna)

```
git clone <repozytorium> universus
cd universus/client
pnpm install
pnpm dev            # buduje WASM, startuje Vite (zwykle http://localhost:5173)
```

`pnpm dev` buduje `public/main.wasm` i uruchamia serwer Vite. Vite obserwuje `sim/` i `data/` i przebudowuje WASM po każdej zmianie plików `.go` lub `.json`, a potem przeładowuje stronę. Bez serwera Go działają tryby lokalne: versus na jednej klawiaturze, versus AI i trening.

10.3 Gra przez sieć w trybie deweloperskim

```
# terminal 1
cd client && pnpm relay      # = go run ../server  (port 8080, bez bazy)
# terminal 2
cd client && pnpm dev        # Vite przekazuje /ws do localhost:8080
```

Obaj gracze otwierają ten sam adres z tym samym kodem pokoju, np. `http://localhost:5173/?room=ABC`. Vite przekazuje do serwera tylko `/ws`, nie `/api`. Konta i ranking wymagają więc trybu z sekcji 10.5. Uwaga: serwer z bazą wymaga zalogowania także do gry po kodzie (D104), dlatego w tym trybie uruchamiaj relay bez `DATABASE_URL`.

10.4 Baza danych (opcjonalnie)

```
docker run --rm -e POSTGRES_PASSWORD=dev -e POSTGRES_DB=universus \
    -p 5432:5432 postgres:16-alpine

export DATABASE_URL=postgres://postgres:dev@localhost:5432/universus
export UNIVERSUS_JWT_SECRET=<długi losowy ciąg>
```

Migracje uruchamiają się same przy starcie serwera. Bez `UNIVERSUS_JWT_SECRET` serwer podpisuje tokeny losowym kluczem, więc każda sesja kończy się wraz z procesem.

10.5 Pełna wersja z jednego portu (jak na produkcji)

```
cd client && pnpm build                       # -> client/dist (z main.wasm)
cd ..
DATABASE_URL=... UNIVERSUS_JWT_SECRET=... \
    go run ./server -static client/dist -addr :8080
# otwórz http://localhost:8080
```

Serwer podaje zbudowanego klienta i WebSocket na jednym originie (D93). Właśnie tego potrzebuje sesja zdalna: jeden port do wystawienia przez tunel, a gość na https może otworzyć socket do tego samego hosta. Po zmianie danych postaci trzeba zrestartować serwer, bo dane są wbudowane w binarkę.

10.6 Testy lokalnie

```
go vet ./... && go test ./...
tools/determinism.sh
node tools/bench.mjs
cd client && pnpm lint && pnpm exec vitest run && pnpm build

# testy z prawdziwym Postgresem
docker run --rm -e POSTGRES_PASSWORD=dev -e POSTGRES_DB=universus -p 55432:5432 postgres:16-alpine
TEST_DATABASE_URL=postgres://postgres:dev@localhost:55432/universus go test ./server/
```

10.7 Parametry URL

Parametr	Działanie
(brak)	Menu: ekran tytułowy i wybór postaci.
`?training`	Tryb treningowy: nieskończone zasoby, bez zegara, nakładka boxów, manekin.
`?ai` lub `?ai=easy|normal|hard`	Walka z AI na wybranym poziomie (domyślnie normal).
`?p1=0&p2=1`	Wybór postaci po indeksie w rosterze. Obaj gracze online muszą podać tę samą parę.
`?room=KOD`	Gra online z drugą osobą z tym samym kodem.
`?bot`	Zamiast klawiatury deterministyczny bot wejść (do pomiarów).
`?boxes`	Nakładka hitboxów i hurtboxów poza treningiem.
`?sweep`	Strona pomiarowa: czas klatki przy rollbacku 0–8.
`?bench`	Pozostawia symulację nietkniętą dla zewnętrznego narzędzia pomiarowego.

10.8 Sterowanie

Akcja	Gracz 1	Gracz 2
Kierunki	W A S D	Strzałki
LP / MP / HP	U / I / O	Numpad 7 / 8 / 9
LK / MK / HK	J / K / L	Numpad 4 / 5 / 6
Rzut (LP+LK)	H	Numpad 0
Gamepad	standardowe mapowanie: ciosy w górnym rzędzie, kopnięcia w dolnym	

Notacja ruchów używa klawiatury numerycznej przy postaci patrzącej w prawo: 6 = do przodu, 4 = do tyłu, 2 = dół, 236 = (QCF), 623 = (DP), 214 = (QCB).

11. Typowe zadania

11.1 Dodanie lub zmiana ruchu

- Edytuj plik postaci w `data/characters/`. Loader od razu zgłosi błędne pole.
- Uruchom `python3 tools/sprites.py`, żeby tabela ruch tag i placeholdery pasowały do listy ruchów (CI tego pilnuje).
- Uruchom `go test ./data ./sim`. Jeśli dodajesz combo, test na prawdziwym rosterze sprawdzi, czy się łączy.
- Zmiana danych zmienia hash `Version()`: starsze buildy nie połączą się z nowym. Zrestartuj serwer.

11.2 Dodanie postaci

- Utwórz `data/characters/NN-nazwa.json` (kolejny numer = indeks w rosterze) z pełnym zestawem normali w obu postawach.
- Uruchom `tools/sprites.py`: każda postać dostaje arkusz nazwany jak jej pole `name` małymi literami.
- Testy w `data/` sprawdzą m.in. kompletność normali, rzut i trzy supery.

11.3 Dodanie grafiki

- Rysuj w Aseprite. Tag animacji = `id` ruchu albo nazwa stanu symulacji (D113).
- Każdy eksport potrzebuje slice'a `origin` wskazującego stopy (D117). Eksportuj jako JSON Array do `art/<postać>/`, nigdy do `client/public/sprites/`.
- Czasy trwania klatek z Aseprite stają się klatkami symulacji, więc suma powinna odpowiadać długości ruchu.
- Uruchom `tools/sprites.py`. Narysowane tagi zastępują placeholdery jeden po drugim.

11.4 Po znalezieniu desync

- Zapisz log wejść z sesji przyciskiem zapisu logu w kliencie (pobiera plik `playtest.inputs` w formacie `replaylog`).
- Uruchom `tools/determinism.sh` albo `replay dump`, żeby zobaczyć, które pole stanu się różni.
- Po naprawie dodaj log do `testdata/`, żeby bramka determinizmu pilnowała go na zawsze.

12. Stan projektu

- **M0–M3 gotowe w kodzie**: symulacja, rollback, netcode, trening, AI, harness bot-vs-bot, cały backend (konta, Postgres, kolejki ranked i casual, wyniki meczów, ranking, rozłączenia, weryfikacja przez ponowną symulację, trzy kontrole anty-cheat), menu i wybór postaci.
- **Zmierzone progi M0**: krok symulacji p99 0,023 ms (próg 0,5 ms), rollback 8 klatek p99 0,540 ms (próg 4 ms), 10 001 identycznych checksumów natywnie i w WASM, stałe 60 fps przy ciągłym rollbacku.
- **Otwarty punkt**: pomiar RTT przez prawdziwą sieć między dwoma komputerami. Kod jest gotowy, potrzebny jest partner i tunel.
- **M4 (grafika)**: część inżynierska gotowa (ładowanie eksportów Aseprite, animacja ze snapshotu). Rysowanie jeszcze się nie zaczęło.
- **M5**: macierz pomiarowa, pomiar opóźnienia kamerą, wdrożenie i tekst pracy.

Źródłem prawdy o tym, co i dlaczego budować, jest vault Obsidian (`/home/kyzelm/Obsidian/Universus/`): Decision Log (D1–D121), dziennik implementacji, tablica zadań i notatki architektury. Przed zaproponowaniem alternatywy warto sprawdzić Decision Log, bo większość alternatyw już rozważono i odrzucono z podanym powodem.